

Implementación del Patrón Singleton en la Conexión a la Base de Datos

1. Introducción

En el desarrollo de nuestro sistema de citas médicas, necesitamos gestionar la conexión a la base de datos de manera eficiente. Para evitar múltiples instancias de conexión y asegurar un único punto de acceso, implementamos el patrón Singleton. Este patrón garantiza que solo exista una instancia de la conexión a la base de datos en toda la aplicación, optimizando recursos y mejorando el rendimiento.

2. ¿Qué es el Patrón Singleton?

El patrón Singleton es un patrón de diseño creacional que asegura que una clase tenga solo una instancia y proporciona un punto de acceso global a ella. Es especialmente útil para recursos compartidos, como conexiones a bases de datos, donde múltiples instancias podrían causar problemas de rendimiento o inconsistencia.

3. Participantes del Patrón Singleton

En nuestra implementación, los participantes son:

1. ConexionBD (Singleton):
 - 1.1. Clase que encapsula la conexión a la base de datos.
 - 1.2. Garantiza una única instancia mediante un constructor privado y un método estático (getInstance).
2. Cliente (Main o cualquier otra clase):
 - 2.1. Utiliza ConexionBD.getInstance() para obtener la instancia única de la conexión.

4. Implementación en el Proyecto

```
Clase ConexionBD (Singleton)
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {
    // Atributo estático que guarda la única instancia
```

```

private static ConexionBD instancia;
private Connection conexion;

// Constructor privado para evitar instanciación externa
private ConexionBD() {
    try {
        // Parámetros de conexión (ajustar según tu configuración)
        String url = "jdbc:mysql://localhost:3306/issste";
        String usuario = "tu_usuario";
        String contraseña = "tu_contraseña";

        // Crear la conexión
        this.conexion = DriverManager.getConnection(url, usuario,
contraseña);
        System.out.println("Conexión establecida correctamente.");
    } catch (SQLException e) {
        System.err.println("Error al conectar a la BD: " +
e.getMessage());
    }
}

// Método estático para obtener la instancia única
public static ConexionBD getInstance() {
    if (instancia == null) {
        instancia = new ConexionBD();
    }
    return instancia;
}

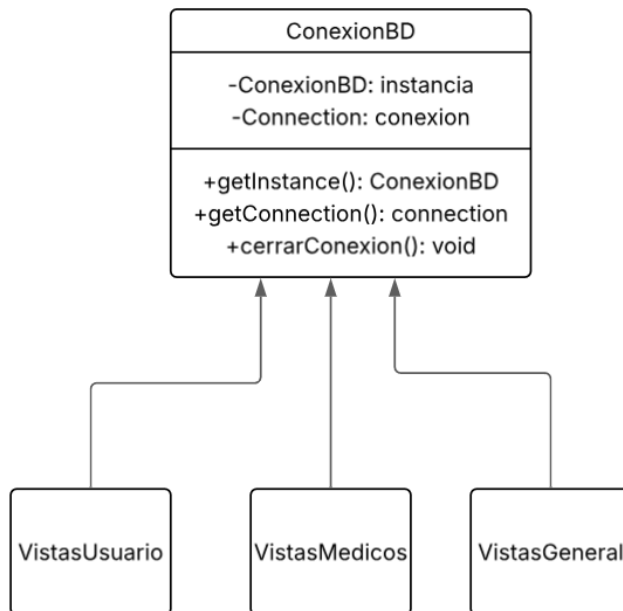
// Método para obtener la conexión
public Connection getConexion() {
    return conexion;
}

// Método para cerrar la conexión (opcional)
public void cerrarConexion() {
    try {
        if (conexion != null && !conexion.isClosed()) {
            conexion.close();
            System.out.println("Conexión cerrada.");
        }
    } catch (SQLException e) {
        System.err.println("Error al cerrar la conexión: " +
e.getMessage());
    }
}

```

}

5. Diagrama UML



6. Uso del Singleton en el Cliente

Clase Main

```
public class Main {
    public static void main(String[] args) {
        // Obtener la instancia única de la conexión
        ConexionBD conexionBD = ConexionBD.getInstance();
        Connection conexion = conexionBD.getConnection();

        // Ejemplo: Usar la conexión en el Facade de autenticación
        FacadeAutenticacion facade = new FacadeAutenticacion(conexion);

        // Operaciones con la BD...
        Usuario usuario = facade.iniciarSesion("correo@example.com",
"contraseña123");

        // Cerrar la conexión al finalizar (opcional)
        conexionBD.cerrarConexion();
    }
}
```

9. Conclusión

El patrón Singleton ha sido clave para gestionar eficientemente la conexión a la base de datos en nuestro sistema. Su implementación asegura consistencia, reduce el overhead y simplifica el mantenimiento del código.